

Lab session 1: Programming Fundamentals

Note: In this first lab session you are only allowed to make use of constructs that are taught in the first week of the course (so no loops, recursion, arrays, functions, etc).

Problem 1: Hello world!

The first exercise is to make a small computer program that outputs `Hello world!` on the screen. The goal of the exercise is to get acquainted with the programming on a Linux environment and the automatic assessment system *Themis*.

1. After logging in to a linux system, open a terminal window. In this terminal you can type commands. Start by creating a directory `progfun` that you will use for the course *Programming Fundamentals*. You create this directory with the command:

```
mkdir progfun
```

2. Navigate to this directory using the command `cd` (change directory).

```
cd progfun
```

3. Create another directory `lab1`, which is a subdirectory of `progfun`. Create in the directory `lab1` another subdirectory `hello` and navigate to this directory:

```
mkdir lab1
cd lab1
mkdir hello
cd hello
```

Create the file `hello.c` using an editor (for example `geany`).

```
geany hello.c &
```

Type the following program, replace `...` by the right information, and save it.

```
/* file:      hello.c                */
/* author:    ..... (email: ....) */
/* date:      .....                */
/* version:   .....                */
/* Description: This program prints 'Hello world!' */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(int argc, char *argv[]) {
    printf("Hello world!\n");
    return 0;
}
```

4. Compile the program using the command:

```
gcc -std=c99 -Wall -pedantic hello.c
```

5. On successful compilation, an executable file `a.out` is produced. Run the program:

```
./a.out
```

6. If you are convinced that your program works well, you can submit it to the online assessment system Themis. You can reach Themis via the link <https://themis.housing.rug.nl> using your favourite webbrowser. You have completed this first task once Themis accepted your submission.

Problem 2: Exam paper

A university professor needs to print a number of exams. Each exam consists of the same number of sheets of paper. The paper used for printing is supplied in packages containing a fixed number of pages. You are asked to write a program that computes the number of packages needed to print all the exams.

The input consists of three integers: the number of exams to print, the number of sheets needed for each exam, and the number of pages in one package.

Make sure that the format of your input and output matches the following examples.

Example 1:

input:

100 5 500

output:

1

Example 2:

input:

200 5 500

output:

2

Example 3:

input:

250 2 100

output:

5

Problem 3: Palindromic number?

A palindromic number is a number that reads the same backward as forward. Examples of palindromic numbers are 11 and 121.

Write a program that accepts as its input an integer between 0 and 999 (inclusive). The output must be YES if the input number is palindromic or can be made palindromic by rearranging its digits, and NO otherwise.

Example 1:

input:

1

output:

YES

Example 2:

input:

42

output:

NO

Example 3:

input:

422

output:

YES

Problem 4: Passing grade

You are requested to write a program that computes a final course grade as a weighted average of three grades. The input consists of six integers: three grades followed by their three corresponding weights. The weights are given as percentages and their sum is 100. You may assume that all grades are between 0 and 10.

Compute the weighted average and round it to the nearest integer or half integer. For example, a grade may be rounded to 6 or 6.5. The grade 5.5 is an exception: it should be rounded to 6.

The output should be the final grade.

Example 1:

input:

10 9 8 20 20 60

output:

8.5

Example 2:

input:

7 6 7 30 30 40

output:

6.5

Example 3:

input:

6 5 6 40 30 30

output:

6

Problem 5: Attacking Queens?

On a chessboard, a queen can move horizontally, vertically, or diagonally.

The input for this programming exercise consists of the positions of two queens in standard chess notation. A position consists of a letter from a to h, indicating the column, followed by a digit from 1 to 8, indicating the row.

The output must be YES if the two queens attack each other, and NO otherwise.

Example 1:

input:

a1 a7

output:

YES

Example 2:

input:

b2 e5

output:

YES

Example 3:

input:

c3 e4

output:

NO